DESIGN & ANALYSIS OF ALGORITHMS

UNIT-4

→ Graph Algorithms
- Graph Traversals DFS & BFS
- Transitive closure
- Directed Acyclic Graphs
- Topological Ordering
- Network Flow Algorithms

→ Tries
- Standard Tries
- Compressed Tries
- Suffix Tries
- Search Engine Indexing

→ External Searching
- (a, b) Trees
- B-Trees

Graph Traversal

A traversal is a systematic procedure for exploring a graph by examining all of its vertices and edges. A traversal is efficient if it visits all the vertices & edges in a linear time. The two traversal methods described are :

(1) **Breadth First Search** : In breadth first search start at a vertex 'v' and mark it as having been visited. The vertex 'v' is at this time said to be unexplored. A vertex is said to have been explored when all vertices adjacent from it has been visited. All unvisited vertices adjacent from 'v' are visited next. These are new unexplored vertices. Vertex 'v' has now been explored. The newly visited vertices haven't been explored and are put onto the end of a list of unexplored vertices. The first vertex on this list is the next to be explored. Exploration continues until no unexplored vertex is left. The list of unexplored vertices operates as a queue and

can be represented using standard representations.

```
1  Algorithm BFS(v)
2  // A breadth first search of G is carried out beginning
3  // at vertex v. For any node i, visited[i] = 1 if i has
4  // already been visited. The graph G and array
5  // visited[] are global; visited[] is initialized to zero.
6  {
7      u := v; // q is a queue of unexplored vertices.
8      visited[v] := 1;
9      repeat
10     {
11         for all vertices w adjacent from u do
12         {
13             if (visited[w] = 0) then
14             {
15                 Add w to q; // w is unexplored.
16                 visited[w] := 1;
17             }
18         }
19         if q is empty then return; // No unexplored vertex.
20         Delete the next element, u, from q;
21                                  // Get first unexplored vertex.
22     } until (false);
```

(2) Depth First Search : In depth first search the exploration of a vertex 'v' is suspended as soon as a new vertex is reached. At this time the exploration of the new vertex 'u' begins. When this new vertex has been explored, the exploration of 'v' continues. The search terminates when all reached vertices have been fully explored.

```
1  Algorithm DFS(v)
2  // Given an graph G = (V, E) with n vertices
3  // and an array visited[]. Initially set
4  // to zero, all the vertices reachable from
5  // v are visited. G and visited[] are global
6  {
7       visited[v] := 1;
8       for each vertex w adjacent from v do
9       {
10        if (visited[w] = 0) then DFS(w);
11      }
12 }
```

Connected Components and Spanning Trees

If G is a connected undirected graph, then all vertices of G will get visited on the first call of BFS. If G is not connected, then atleast two calls to BFS will be needed. Hence, BFS can be used to determine whether G is connected.

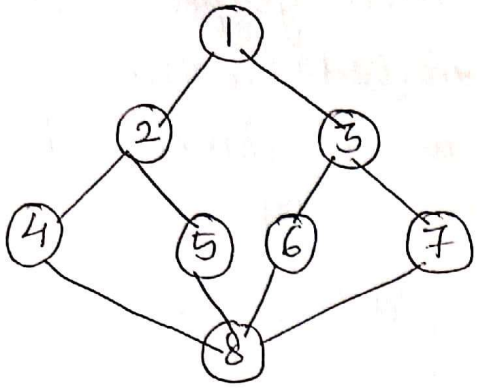
All newly visited vertices on a call to BFS from BFT represent the vertices in a connected component of G. Hence the connected components of a graph can be obtained using BFT.

As a final application of BFS or DFS, is the problem of obtaining a spanning tree for an undirected graph G.

The graph G has a spanning tree iff G is connected. Hence, BFS / DFS determines the existence of a spanning tree.

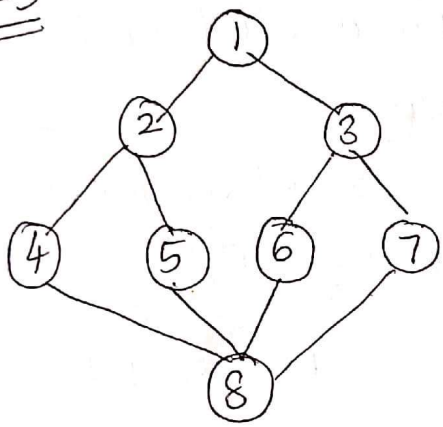
Spanning trees obtained using a BFS/DFS are called breadth / depth first spanning trees.

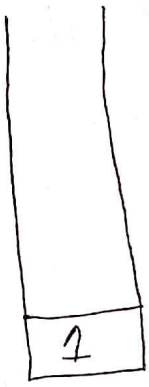
Example : Consider the graph below :



Draw the spanning trees of the graph. (DFS/BFS)

DFS



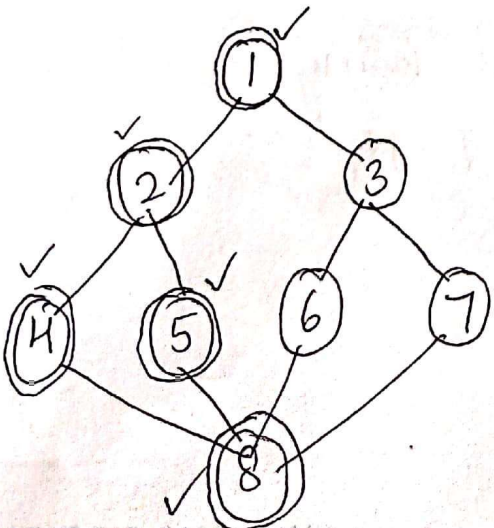


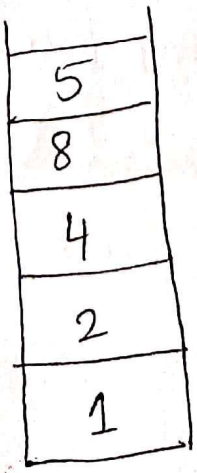
stack

visited [ ] =

1	2	4	8	5	6	3	7

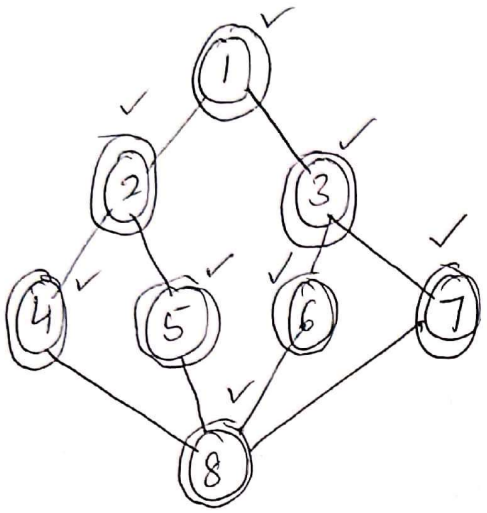
Add 1 to stack. Mark it as visited.
Now look for adjacent vertices that are unvisited and add them to the stack

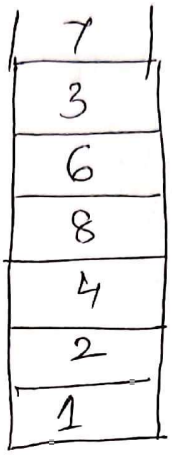




stacks

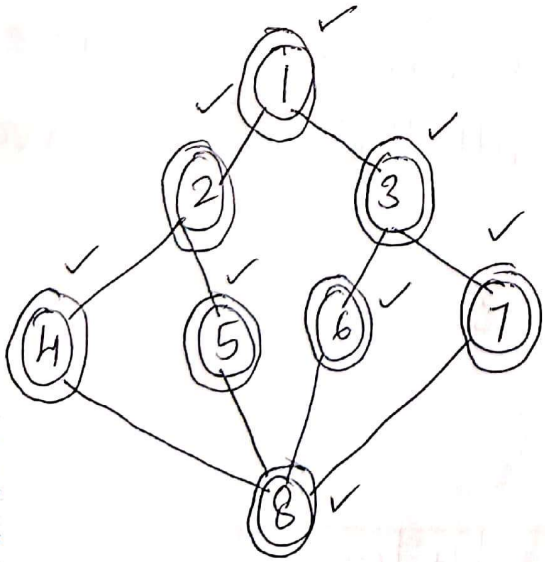
vertex 5 is searched for adjacent unvisited vertices. If no unvisited vertices found. Pop 5 from the stack and look for the unvisited vertices of the top of the stack

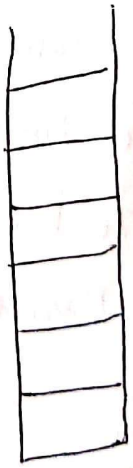




Stack

Pop 7 from the stack and look for the unvisited vertices. If none found. Pop the next element and search. Keep doing until the stack is empty.



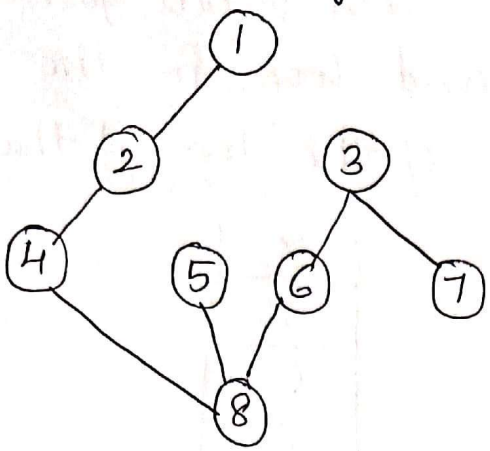


empty stack

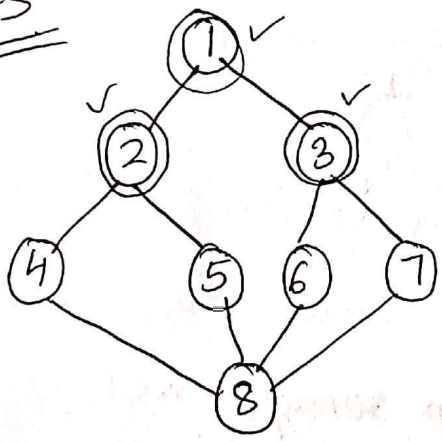
The DFS traversal is:

$1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 5 \rightarrow 6 \rightarrow 3 \rightarrow 7$

The DFS spanning tree is :↓



BFS

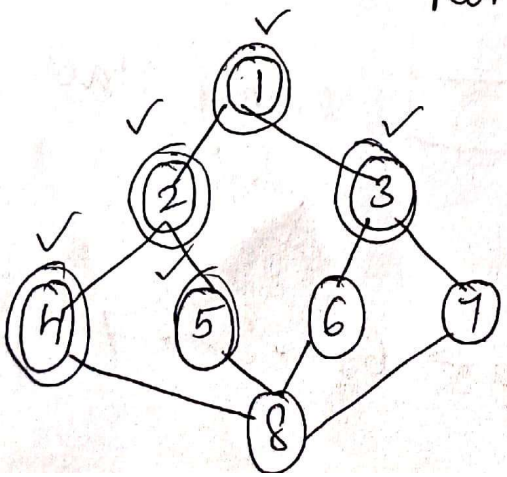


Queue

1	2	3					

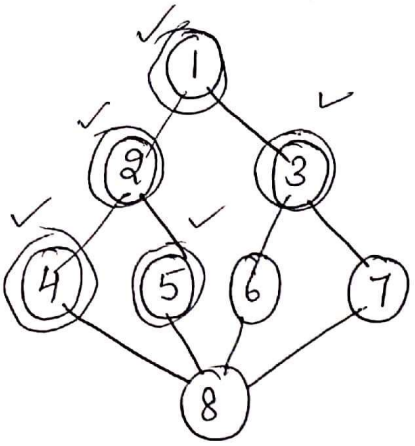
Add a vertex into the queue and mark it as visited. Now add the adjacent vertices of the vertex in queue that are unvisited. Remove the vertex from queue if all the adjacent vertices have been visited.

Remove / Dequeue 1



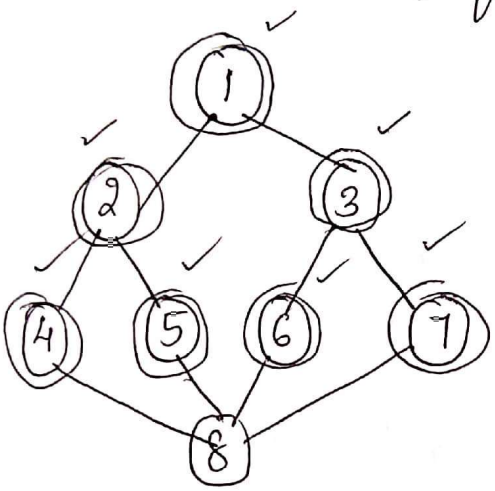
2	3	4	5				

Dequeue 2



2	4	5	6	7	

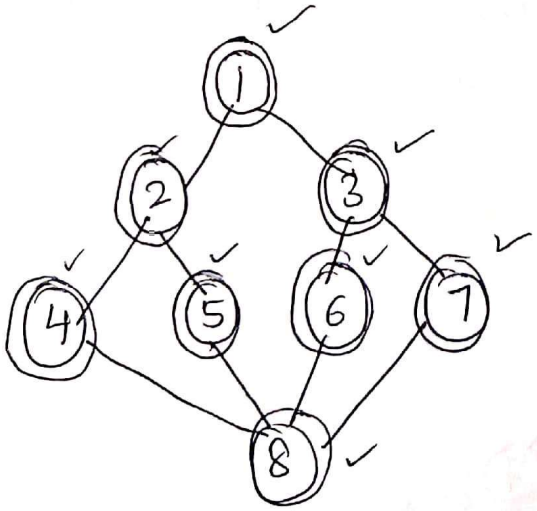
Dequeue 3



4	5	6	7	8	

Dequeue 4, 5, 6, 7, 8. Since all the vertices have been visited.

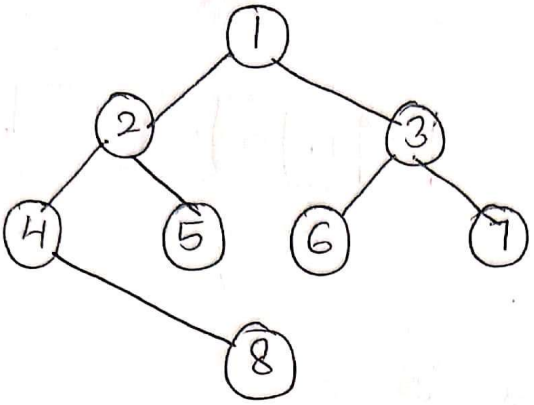
Queue is empty



The BFS traversal is :

1 → 2 → 3 → 4 → 5 → 6 → 7 → 8

The BFS Spanning tree is :↓



Transitive Closure

A directed graph, also known as a digraph, is a graph whose edges are all directed.

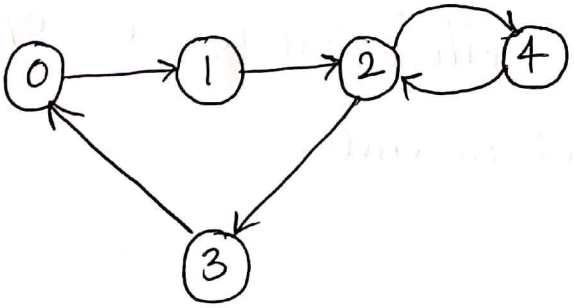
Given vertices 'u' and 'v' of a digraph $\vec{G}$, 'u' reaches 'v' (and 'v' is reachable from 'u') if $\vec{G}$ has a directed path from 'u' to 'v'.

→ A digraph $\vec{G}$ is strongly connected if, for any two vertices 'u' and 'v' of $\vec{G}$, 'u' reaches 'v' and 'v' reaches 'u'.

→ A directed cycle of $\vec{G}$ is a cycle where all the edges are traversed according to their respective directions.

→ A digraph $\vec{G}$ is acyclic if it has no directed cycles.

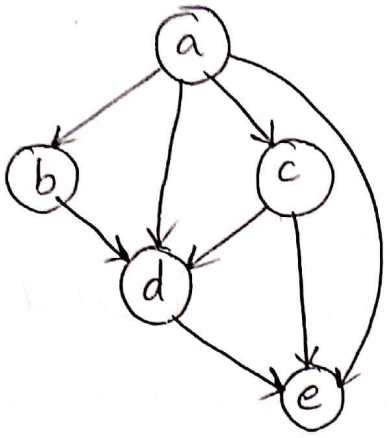
Example :



Strongly connected digraph

directed cycle : →

0 → 1 → 2 → 4 → 3 → 0



← Directed acyclic graph

→ The 'transitive closure' of a digraph $\vec{G}$ is the digraph $\vec{G}^*$ such that the vertices of $\vec{G}^*$ are the same as the vertices of $\vec{G}$, and $\vec{G}^*$ has an edge (u, v), whenever $\vec{G}$ has a directed path from 'u' to 'v'. That is, $\vec{G}^*$ is defined by starting with the digraph $\vec{G}$ and adding in an extra edge (u, v) for each 'u' and 'v', such that 'v' is reachable from 'u'.

Let $\vec{G}$ be a digraph with 'n' vertices and 'm' edges. The transitive closure of G is computed in a series of rounds.

- Initialize $\vec{G}_0 = \vec{G}$.
- Number the vertices of $\vec{G}$ as

$$v_1, v_2, \ldots, v_n$$

- Begin the computation of the rounds, beginning with round 1.
- In a generic round k, construct digraph $\vec{G}_k$ starting with $\vec{G}_k = \vec{G}_{k-1}$ and adding to $\vec{G}_k$ the directed edge (v_i, v_j) if digraph $\vec{G}_{k-1}$ contains both the edges (v_i, v_k) and (v_k, v_j).

→ For $i = 1, \ldots, n$, digraph $\vec{G}_k$ has an edge (v_i, v_j) if and only if digraph $\vec{G}$ has a directed path from v_i to v_j, whose intermediate vertices (if any) are in the set $\{v_1, \ldots, v_k\}$. In particular, $\vec{G}_n$ is equal to $\vec{G}^*$, the transitive closure of $\vec{G}$. This suggests a simple dynamic programming algorithm known as "Floyd-Warshall" algorithm can be used to compute the transitive closure of $\vec{G}$.

```
1 Algorithm FloydWarshall(G)
2 // Input : A digraph G with n vertices.
3 // Output : The transitive closure G* of G.
4 // let v1, v2, ... vn be numbering of the vertices
5 // of G
6 {
7     G0 := G;
```

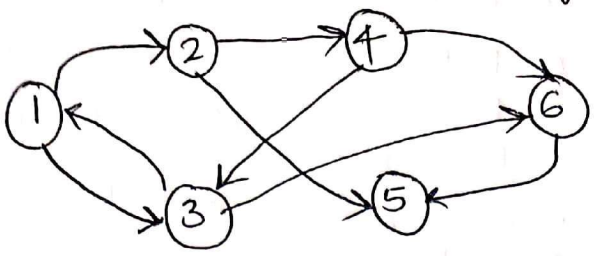
```
8     for k := 1 to n do
9       G→k := G→k-1 ;
10      for i := 1 to n do // i≠k.
11        for j := 1 to n do  //j≠i.
12          if both edges (vi, vk) and (vk, vj) are
13                in G→k-1 then
14            if G→k does not contain directed edge
15                (vi, vj) then
16              add directed edge (vi, vj) to G→k.

17    return G→n.
18  }
```

This dynamic programming algorithm computes the transitive closure G^* of G by incrementally computing a series of digraphs $\vec{G}_0, \vec{G}_1, \ldots, \vec{G}_n$, for $k = 1, \ldots, n$.

→ Let $\vec{G}$ be a digraph with 'n' vertices represented by the adjacency matrix. The Floyd-Warshall algorithm computes the transitive closure $\vec{G}^*$ of $\vec{G}$ in $O(n^3)$ time.

Example : Consider the following graph $\vec{G}$:



$V = \{1, 2, 3, 4, 5, 6\}$ and $E = \{(1,2)\ (1,3)\ (2,4)\ (2,5)\ (3,1)\ (3,6)\ (4,6)\ (4,3)\ (6,5)\}$

Compute the Transitive closure $\vec{G}^*$.

Solution :

Adjacency Matrix for $\vec{G}$

	1	2	3	4	5	6
1	0	1	1	0	0	0
2	0	0	0	1	1	0
3	1	0	0	0	0	1
4	0	0	1	0	0	1
5	0	0	0	0	0	0
6	0	0	0	0	1	0

$T^{(0)}$:

	1	2	3	4	5	6
1	1	1	1	0	0	0
2	0	1	0	1	1	0
3	1	0	1	0	0	1
4	0	0	1	1	0	1
5	0	0	0	0	1	0
6	0	0	0	0	1	1

$T^{(1)}$:

	1	2	3	4	5	6
1	1	1	1	0	0	0
2	1	1	0	1	1	0
3	0	1	0	1	1	0
4	1	1	1	0	0	1
5	0	0	1	1	0	1
6	0	0	0	0	1	0

$3 \rightarrow 2$ through 1

$T^{(2)}$:

	1	2	3	4	5	6
1	1	1	1	1	1	0
2	0	1	0	1	1	0
3	1	1	1	1	1	1
4	0	0	1	1	0	1
5	0	0	0	0	1	0
6	0	0	0	0	1	1

$1 \rightarrow 4$ through 2
$1 \rightarrow 5$ through 2
$3 \rightarrow 4$ through 2
$3 \rightarrow 5$ through 2

$T^{(3)}$:

	1	2	3	4	5	6
1	1	1	1	1	1	1
2	0	1	0	1	1	0
3	1	1	1	1	1	1
4	1	1	1	1	1	1
5	0	0	0	0	1	0
6	0	0	0	0	1	1

$1 \rightarrow 6$ through 3
$4 \rightarrow 1$ through 3
$4 \rightarrow 2$ through 3
$4 \rightarrow 5$ through 3

$T^{(4)}$:

	1	2	3	4	5	6
1	1	1	1	1	1	1
2	1	1	1	1	1	1
3	1	1	1	1	1	1
4	1	1	1	1	1	1
5	0	0	0	0	1	0
6	0	0	0	0	1	1

$2 \rightarrow 1$ through 4
$2 \rightarrow 3$ through 4
$2 \rightarrow 6$ through 4

$T^{(5)}$:

	1	2	3	4	5	6
1	1	1	1	1	1	1
2	1	1	1	1	1	1
3	1	1	1	1	1	1
4	1	1	1	1	1	1
5	0	0	0	0	1	0
6	0	0	0	0	1	1

$T^{(6)}$:

	1	2	3	4	5	6
1	1	1	1	1	1	1
2	1	1	1	1	1	1
3	1	1	1	1	1	1
4	1	1	1	1	1	1
5	0	0	0	0	1	0
6	0	0	0	0	1	1

Transitive closure

Directed Acyclic Graphs

Directed graphs without directed cycles are encountered in many applications. Such a digraph is often referred to as a "directed acyclic graph" or "dag".

Applications of such graphs include the following:

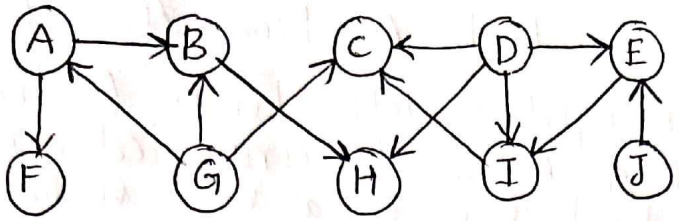
- Inheritance between C++ classes or Java interfaces
- Prerequisites between courses of a degree program
- Scheduling constraints between the tasks of a project.

Topological Ordering

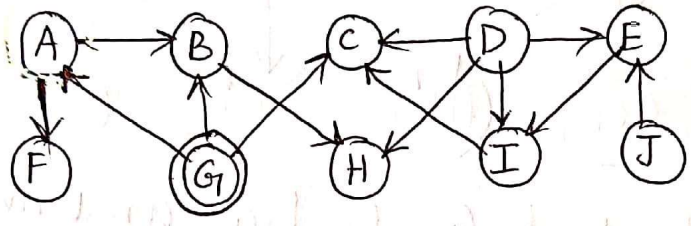
A topological ordering of $\vec{G}$ is an ordering $(v_1, v_2, \ldots, v_n)$ of the vertices of $\vec{G}$ such that for every edge (v_i, v_j) of $\vec{G}$, $i < j$. That is, a topological ordering is an ordering such that any directed path in $\vec{G}$ traverses vertices in increasing order. A digraph may have more than one topological ordering. A digraph has a topological ordering if and only if it is acyclic.

```
1 Algorithm TopologicalSort(G⃗):
2 // Input : A digraph G⃗ with n vertices.
3 // Output : A topological ordering v1, ..., vn of
      G⃗ or an indication that G⃗ has a directed cycle.
4 // Let S be an initially empty stack
5 // for each vertex u of G⃗ do
6     incounter(u) := indeg(u)
7     if (incounter(u) := 0) then
8       S.push(u);
9     i := 1;
10    while S is not empty do
11      u := S.pop();
12      number u as the i-th vertex vi
13      i := i+1;
14      for each edge e ∈ G⃗ : outIncidentEdges(u) do
15        w := G.opposite(u,e)
16        incounter(w) := incounter(w) - 1;
17        if (incounter(w) = 0) then
18          S.push(w);
19        if S is empty then
20          return "digraph G⃗ has a directed cycle"
```

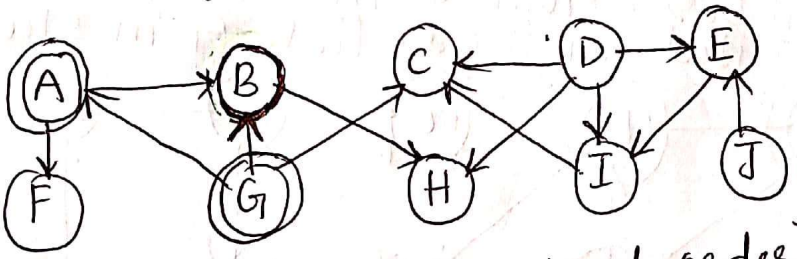
Example : Obtain the topological ordering/sorting of vertices in a given digraph :



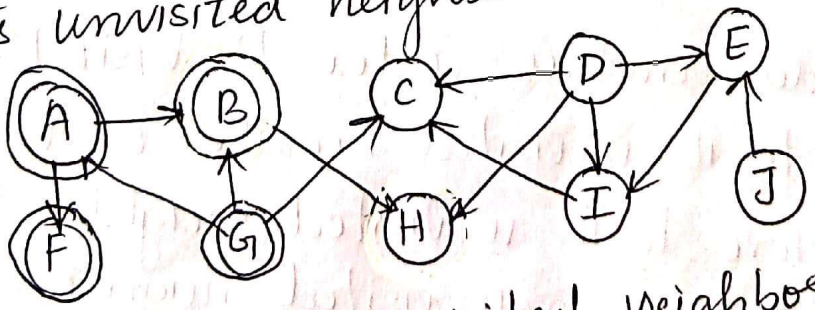
Solution :



Begin, vertex G → it is unvisited
Once visited, mark it visited and look for its unvisited neighbors (A, B, F)

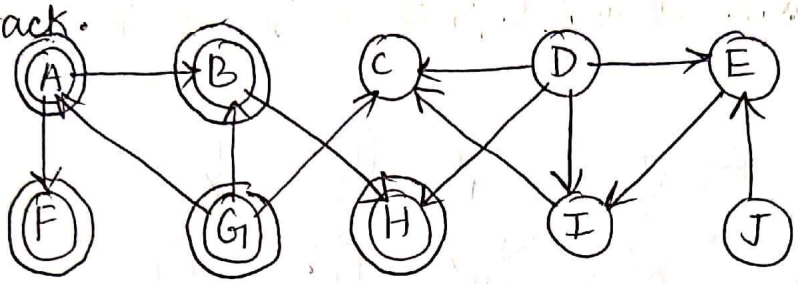


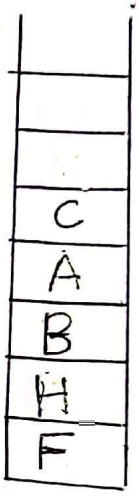
Consider A now (in alphabetical order) look for its unvisited neighbors.



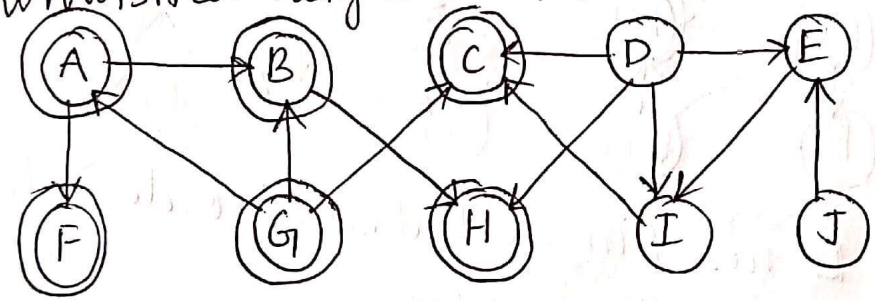
Visit F, look for unvisited neighbors. Since F has no unvisited neighbors. Push F in stack and move back to node A. A has B as unvisited neighbors.

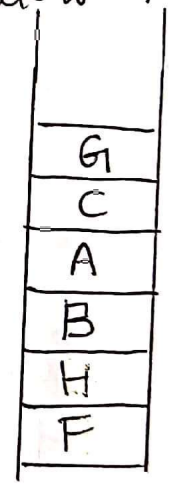
. Visit B. It has H as unvisited neighbors. Push H to stack and move back to B. Now B also has no unvisited neighbors left. Push B in the stack. Move back to A. A has no unvisited neighbors. Push A in stack.



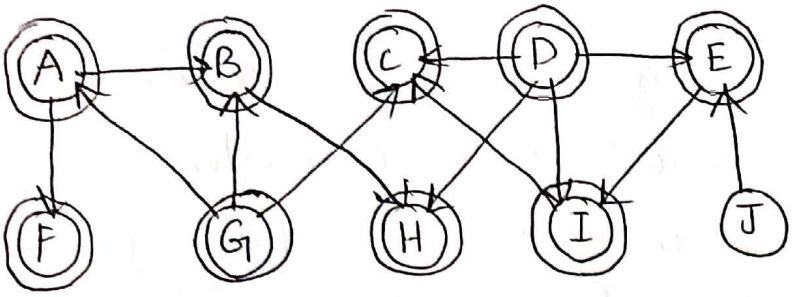


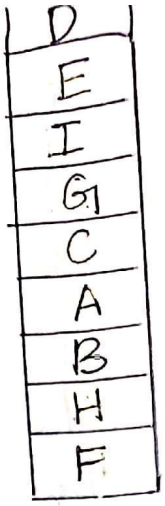
Move back to G. G has an unvisited neighbor C left.
Now consider another unvisited node C. Visit C. Since it has no neighbors. Push C in stack. Move back to G. No unvisited neighbors. Push G in stack.

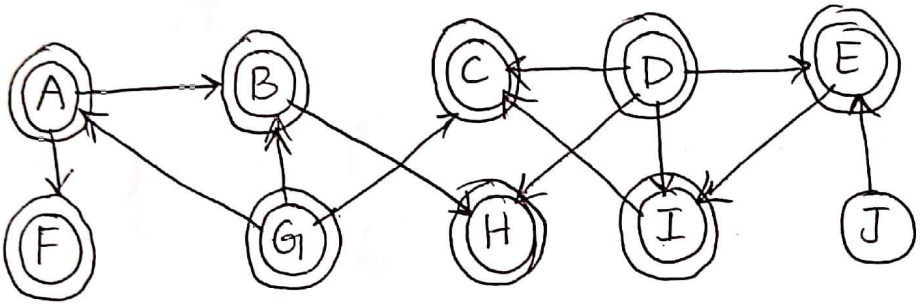


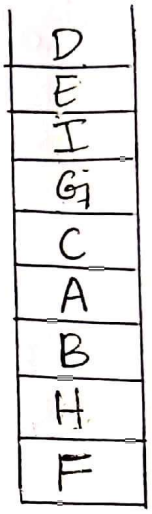


Consider another vertex D. Visit D. Move to its unvisited neighbor E. Visit E. Move to its unvisited neighbor I. Visit I. Since I has no unvisited neighbors. Push I in stack and move back to vertex E. E has no unvisited neighbor. Push E in stack and move back to D. D also has no unvisited neighbors left. Push D in stack.

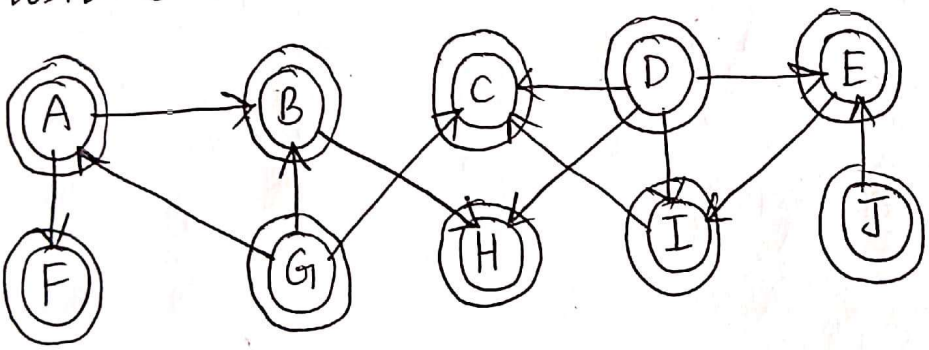


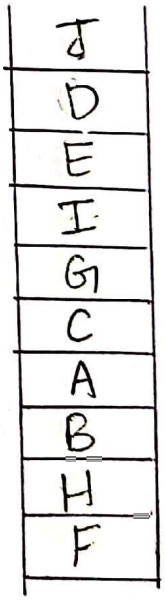






Now choose vertex J. Visit J. Since all its neighbors are already visited. Push J in stack.





Since all the vertices of the digraph are visited. Pop out the stack elements for the Topological ordering of the graph.

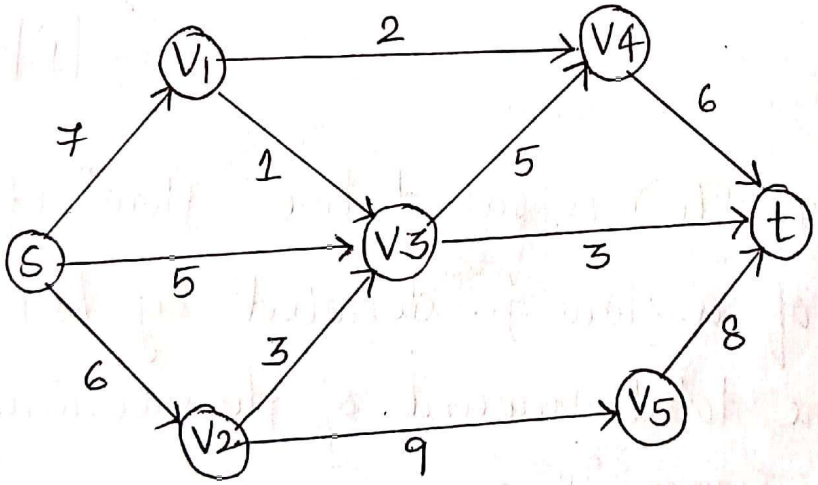
Topological Ordering : J D E I G C A B H F

→ Let $\vec{G}$ be a digraph with 'n' vertices and 'm' edges. The topdogical sorting algorithm runs in $O(n+m)$ time using $O(n)$ auxiliary space, and either computes a topological ordering of $\vec{G}$ or fails to number some vertices, which indicates that $\vec{G}$ has a directed cycle.

Network Flow

A flow network N consists of the following:

- A connected directed graph G with nonnegative integer weights on the edges, where the weight of an edge e is called the capacity c(e) of e.
- Two distinguished vertices, 's' and 't', of G, called the 'source' and 'sink', respectively, such that 's' has no incoming edges and 't' has no outgoing edges.



A 'flow' for network N is an assignment of an integer value f(e) to each edge e of G that satisfies the following properties:

- For each edge e of G,

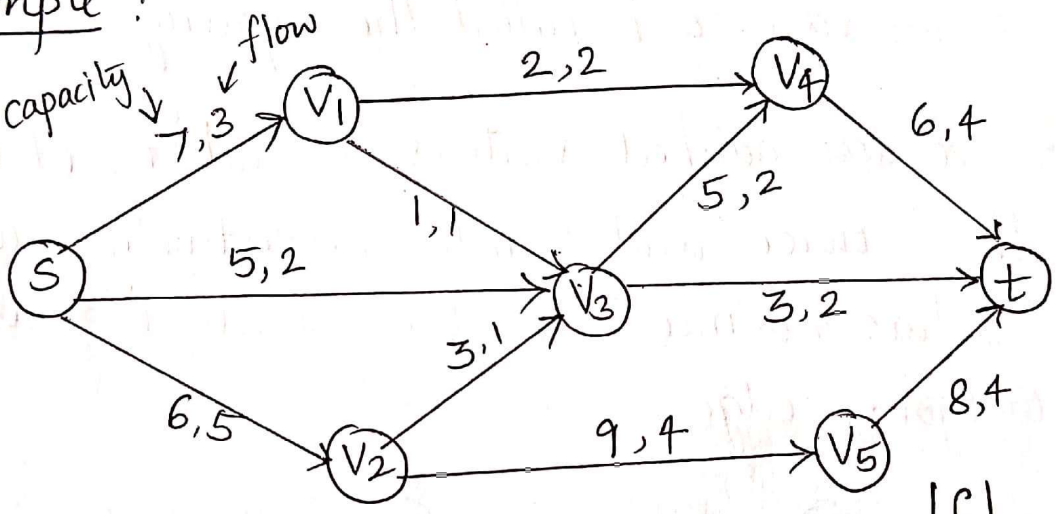
$$0 \leq f(e) \leq c(e) \quad \text{(capacity rule)}$$

- For each vertex v of G distinct from the source s and the sink t

$$\sum_{e \in E^-(v)} f(e) = \sum_{e \in E^+(v)} f(e) \quad \text{(conservation rule)}$$

where $E^-(v)$ and $E^+(v)$ denote the sets of incoming and outgoing edges of v, respectively.

Example:



$|f| = 10$

The quantity $f(e)$ is called the 'flow' of edge e. The value of a flow f, denoted by $|f|$, is equal to the total amount of flow coming out from the source 's':

$$|f| = \sum_{e \in E^+(s)} f(e)$$

A maximum flow for flow network N is a flow with maximum value over all flows for N.

→ Residual Capacity

Let N be a flow network, which is specified by a graph G, capacity function c, source s, and sink. Let 'f' be a flow for N. Given an edge e of G directed from vertex 'u' to vertex 'v', the residual capacity from 'u' to 'v' w.r.t the flow 'f', denoted $\Delta f(u,v)$, is defined as

$$\Delta f(u,v) = c(e) - f(e)$$

and the residual capacity from 'v' to 'u' is defined as

$$\Delta f(v,u) = f(e)$$

Let π be a path from 's' to 't' that is

allowed to traverse edges in either the forward or backward direction. A 'forward edge' of π is an edge e of π such that, in going from s to t along path π, the origin of e is encountered before the destination of e. An edge of π that is not forward is said to be a backward edge.

The residual capacity to an edge e is π traversed from u to v

$$\Delta_f(e) = \begin{cases} c(e) - f(e) & \text{if } e \text{ is a forward edge} \\ f(e) & \text{if } e \text{ is a backward edge} \end{cases}$$

That is, the residual capacity of an edge e going in the forward direction is the additional capacity of e that f has yet to consume, but the residual capacity in the opposite direction is the flow that f has consumed.

→ Augmenting Paths

An augmenting path for flow 'f' is a path π from the source 's' to the sink 't' with nonzero residual capacity, that is, for each edge e of π,

- $f(e) < c(e)$ if e is a forward edge
- $f(e) > c($ if e is a backward edge

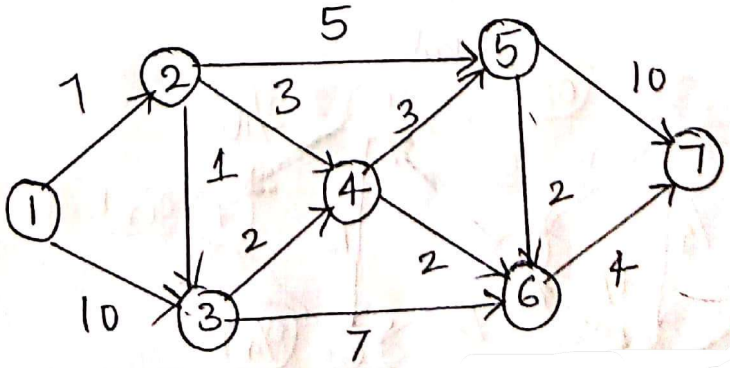
→ The Ford-Fulkerson Algorithm

The main idea of Ford-Fulkerson algorithm is to incrementally increase the value of a flow in stages, where at each stage some amount of flow is pushed along an augmenting path from the source to the sink. Initially, the flow of each edge is equal to 0. At each edge, an augmenting path π is computed and an amount of flow equal to the residual capacity of π is pushed along π. The algorithm terminates when the current flow 'f' does not admit an augmenting path.

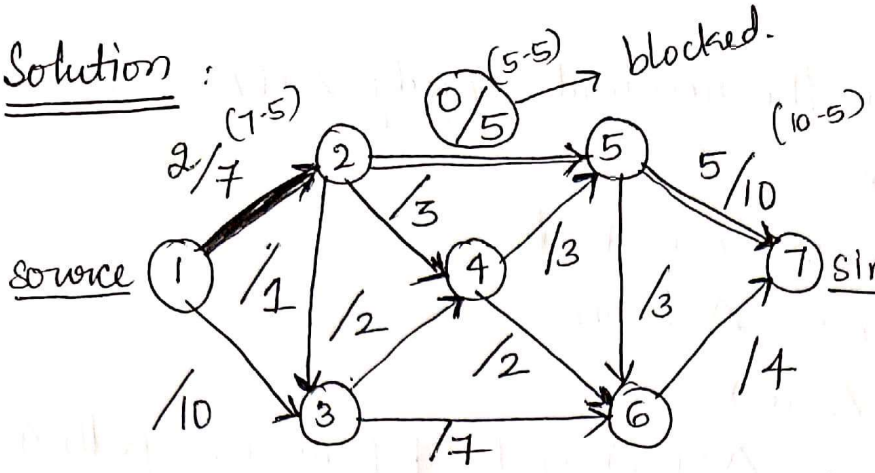
```
1 Algorithm   MaxFlowFordFulkerson (N)
2 Input : Flow network N = (G, c, s, t)
3 Output : A maximum flow f for N
4   for each edge e ∈ N do
5       f(e) ← 0.
6   stop ← false
7   repeat
8       traverse G starting at s to find an augmenting
                                        path for f
9       if augmenting path π exists then
```

```
10  { Compute the residual capacity Δf(π) of π }
11  Δ ← +∞
12  for each edge e ∈ π do
13    if Δf(e) < Δ then
14      Δ ← Δf(e)
15  { Push Δ = Δf(π) units of flow along path π }
16  for each edge e ∈ π do
17    if e is a forward edge then
18      f(e) ← f(e) + Δ
19    else
20      f(e) ← f(e)  Δ   { e is a backward edge }
21  else
22    stop ← true   { f is a maximum flow }
23  until stop
```

Example : Find the maximum flow through the given network using Ford Fulkerson algorithm.



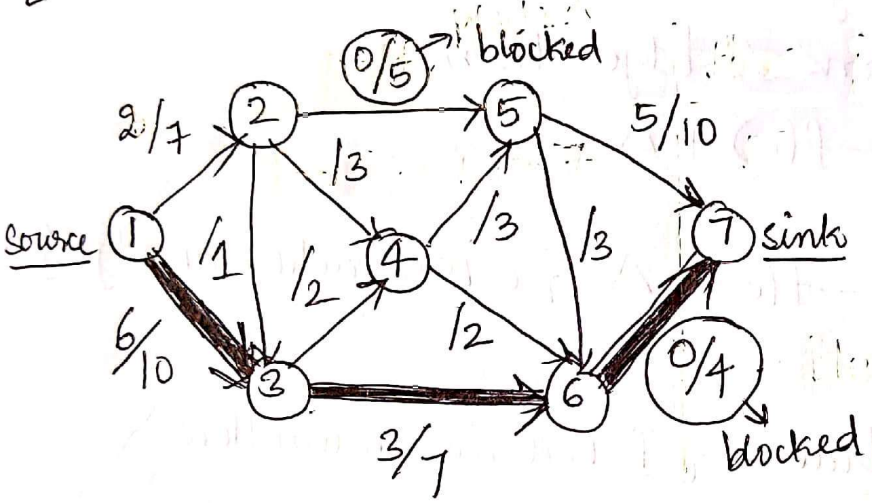
Solution :



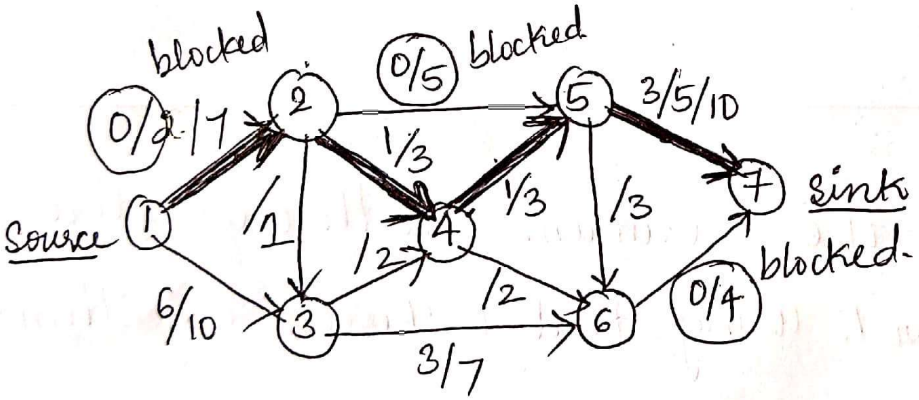
flow ↑ capacity ↗

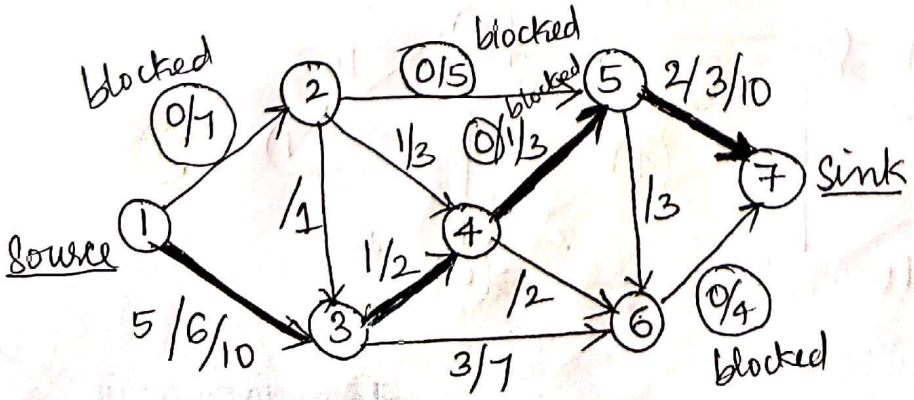
1/5

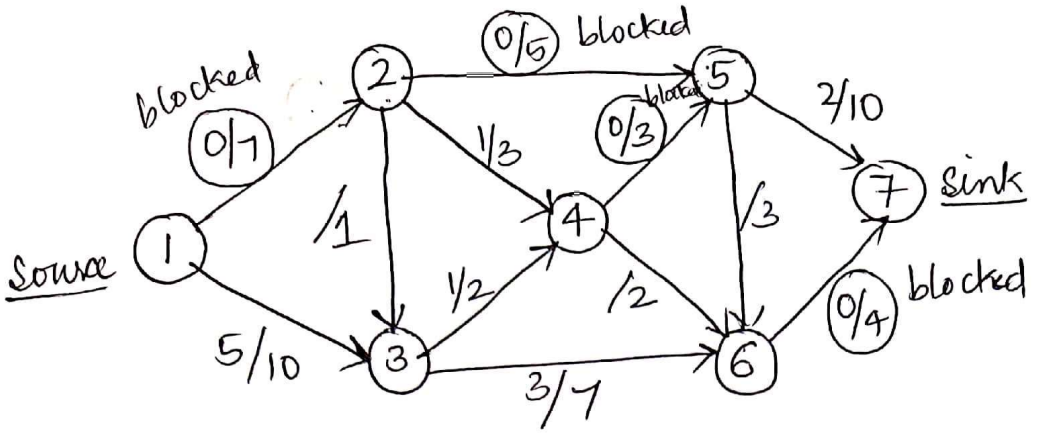
[Choose minimum capacity of the path for flow]



Augmented Path	flow
1→2→5→7	5
1→3→6→7	4
1→2→4→5→7	2
1→3→4→5→7	1







Since there are no more augmented paths left in the network to go from source to sink.

Augmented Paths	Flow
$1 \to 2 \to 5 \to 7$	5
$1 \to 3 \to 6 \to 7$	4
$1 \to 2 \to 4 \to 5 \to 7$	2
$1 \to 3 \to 4 \to 5 \to 7$	1
	12

∴ Maximum flow through the given network using Ford-Fulkerson algorithm is 12.

Tries

All the search trees are used to store the collection of numerical values but they are not suitable for storing the collection of words or strings.

'Trie' is a data structure used to store the collection of strings and makes searching of a pattern in words more easy.

The term 'trie' came from the word retrieval.

Trie data structure makes retrieval of a string from the collection of strings more easily.

Trie is also called as 'Prefix Tree' and 'Digital tree'

Trie is an efficient information storage and retrieval data structure.

A Trie searches a string in O(m) time complexity, where 'm' is the length of the string.

In trie, every node except the root stores a character value. Every node in trie can have one or a number of children. All the children of a node are alphabetically ordered. If any two strings have a common prefix then they will have the same ancestors.

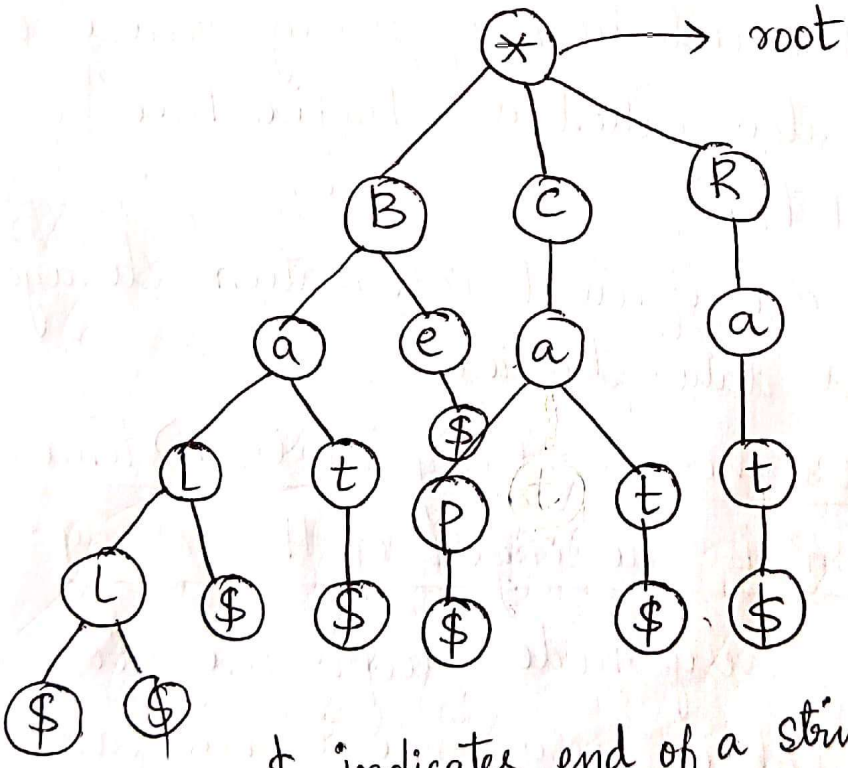
Properties of the Trie for a set of string :

- The root node of the trie always represents the null node.
- Each child node is sorted alphabetically.
- Each node can have a maximum of 26 children
- Each node can store one letter of the alphabet except the root.

Example :

Construct a trie for the following list of strings

Cat , Bat , Ball , Rat , Cap & Be



$ indicates end of a string

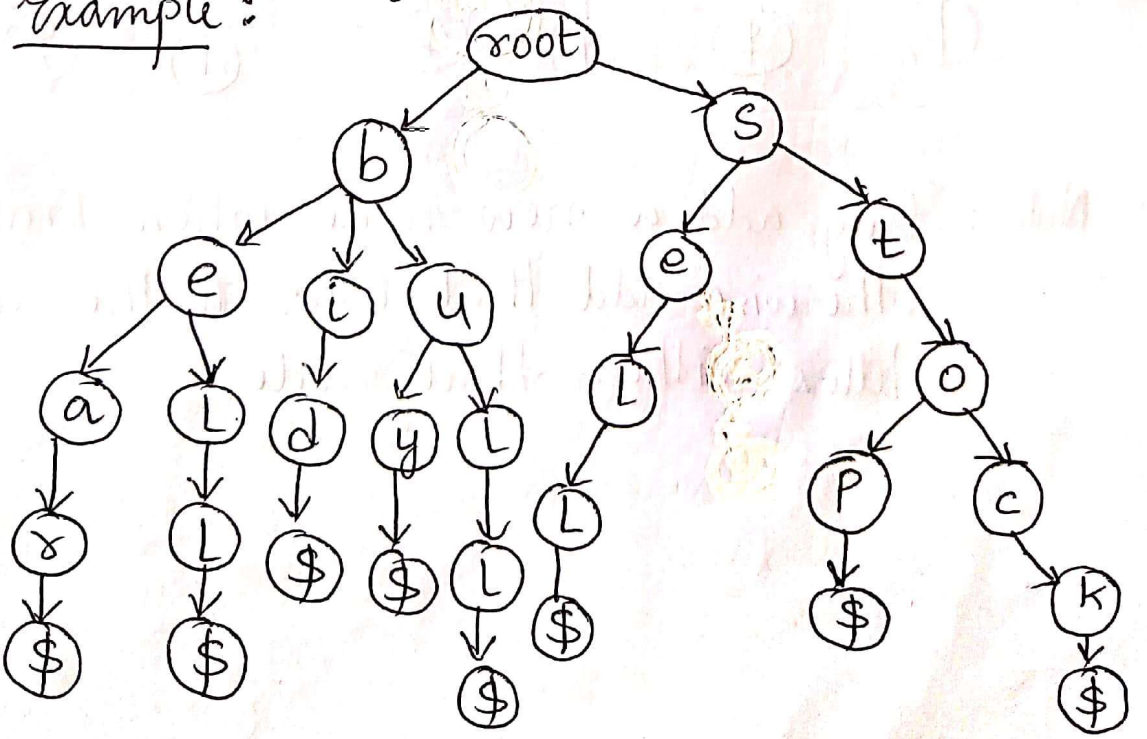
Types of Tries : Tries are classified into three categories :

→ Standard Trie
→ Compressed Trie
→ Suffix Trie

★ **Standard Trie** : It has the following properties:

- It is an ordered tree like data structure
- Each node (except the root node) in a standard trie is labeled with a character.
- The children of a node are in alphabetical order.
- Each node or branch may have multiple branches.
- The last node marks the end of the word.

Example : S = { bear, bell, bid, bull, buy, sell, stock, stop }

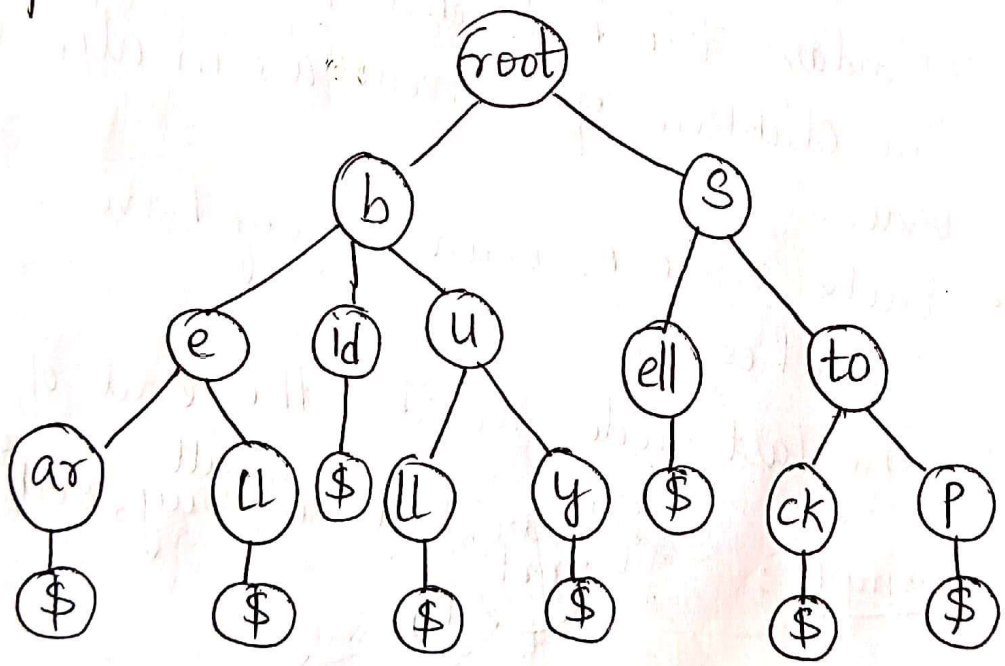


↗ Similar to Standard Trie but ensures that each internal node has atleast two children.

★ **Compressed Trie** : It has the following properties :

- It is an advanced version of standard trie.
- Each nodes (except the leaf nodes) have atleast 2 children.
- It is used to achieve space optimization.
- To derive a compressed Trie from a Standard Trie, compression of chains of redundant nodes is performed.

Example : S = { bear, bell, bid, bull, buy, sell, stock, stop }



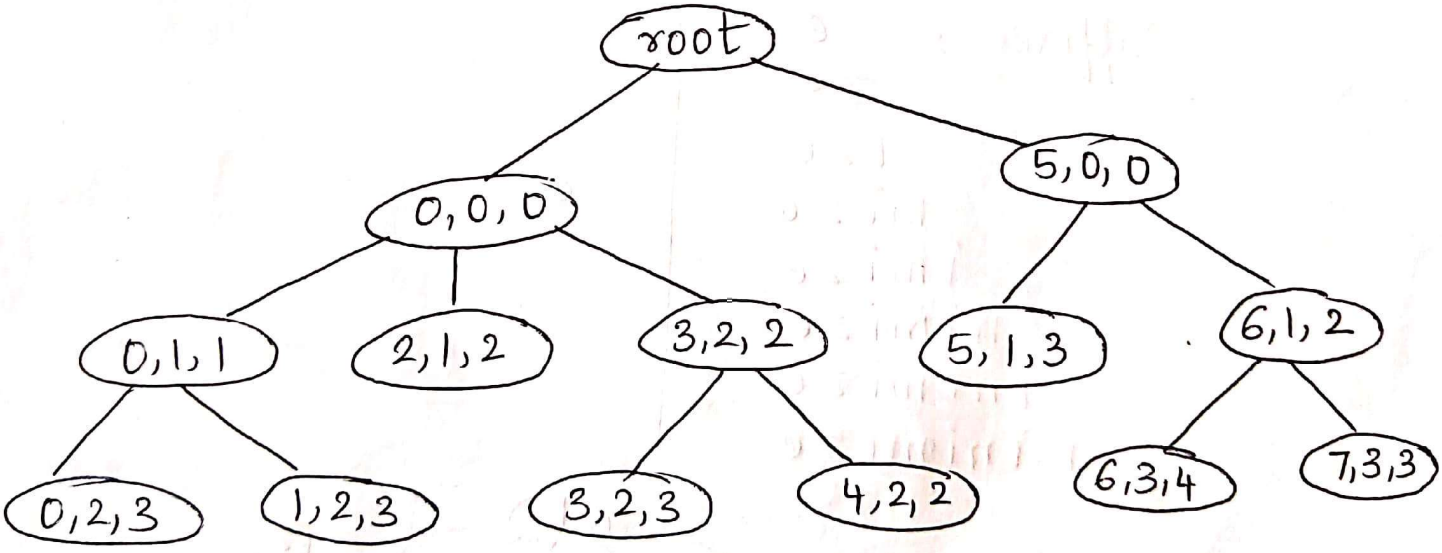
Note : Only add a new node when branching otherwise add that letter to the sequence of letters within that node

Compact Representation of a Compressed Trie

{ bear, bell, bid, bull, buy, sell, stock, stop }

	0	1	2	3	4
s[0] =	b	e	a	r	
s[1] =	b	e	l	l	
s[2] =	b	i	d		
s[3] =	b	u	l	l	
s[4] =	b	u	y		
s[5] =	s	e	l	l	
s[6] =	s	t	o	c	k
s[7] =	s	t	o	p	

It is represented by a triplet of integers (i, j, k), such that i → index of s
j → start of prefix
k → end of prefix.



Strings in the collection S are all the suffixes of string X.

★ **Suffix Trie** : It has the following properties:

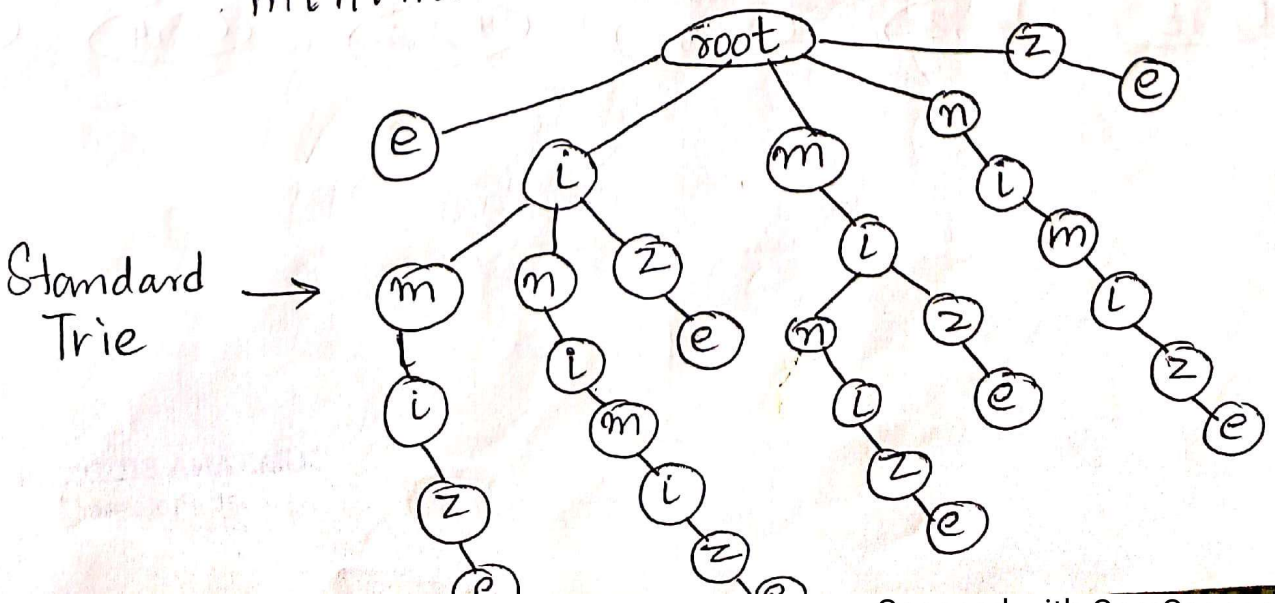
- It is an advanced version of the compressed trie.
- It is used in pattern matching, word matching and prefix matching.
- To generate a suffix trie, all the suffixes of given string are considered as individual words.
- Using the suffixes, compressed trie is built.

Example : minimize

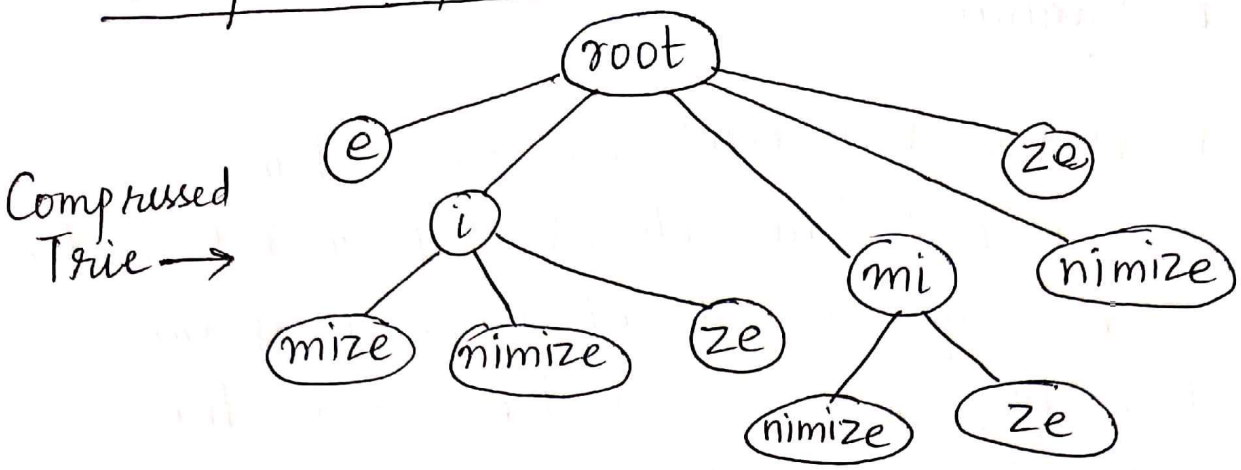
① Identify the suffixes of the word

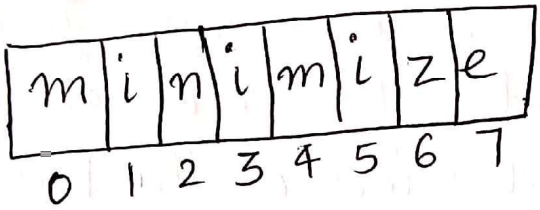
Suffixes :
e
ze
ize
mize
imize
nimize
inimize
minimize

S



Compact Representation of a Suffix Trie

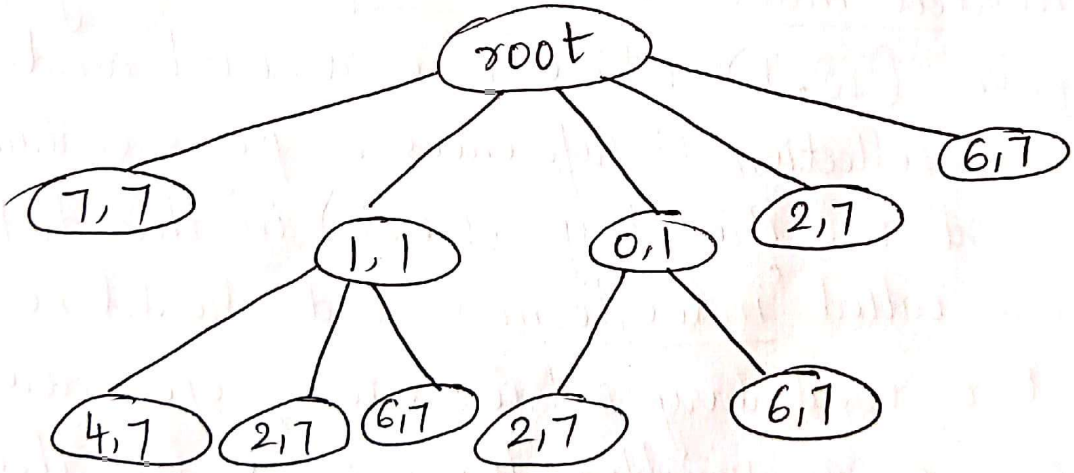




Construct the trie so that the label of each vertex is a pair (i, j) such that

i → start

j → end



Search Engines

The World Wide Web contains a huge collection of text documents (Web pages). Information about these pages is gathered by a program called a "Web crawler", which then stores this information in a special dictionary database. A Web search engine allows users to retrieve relevant information from this database, thereby identifying relevant pages on the Web containing given keywords.

Inverted Files - The core information stored by a search engine is a dictionary, called an "inverted index" or "inverted file", storing key-value pairs (w, L) where w is a word and L is a collection of references to pages containing word w. The keys (words) in this dictionary are called "index terms" and should be a set of vocabulary entries and proper nouns as large as possible. The elements in this dictionary are called "occurrence lists" and should cover as many Web pages as possible.

An inverted index can be efficiently implemented with a data structure consisting

of the following :

- An array storing the occurence lists of the terms in no particular order.
- A compressed trie for the set of index terms, where each external node stores the index of the occurence list of the associated term.

The reason for storing the occurence lists outside the trie is to keep the size of the trie data structure sufficiently small to fit in internal memory. Instead, because of their large total size, the occurence lists have to be stored on disk.

A query for a single keyword is similar to a word matching query, the keyword is searched in the trie and the associated occurence list is returned.

When multiple keywords are given and the desired output is the pages containing all the given keywords, the occurence list of each keyword using the trie is retrieved and their intersection is returned. To facilitate the intersection computation, each occurence list should be implemented with a sequence sorted by address or with a dictionary.

In addition to the basic task of returning

a list of pages containing given keywords, search engines provide an important additional service by ranking the pages returned by relevance. Devising fast and accurate ranking algorithms for search engines is a major challenge for computer researchers and electronic commerce companies.

External Searching

There exists a great time difference between main memory accesses and disk accesses. The main goal of maintaining a dictionary in external memory is to minimize the number of disk transfers needed to perform a query or update.

Since sequence dictionary implementations are inefficient, the logarithmic-time, internal memory strategies using balanced binary trees (AVL, red-black) or other search structures with logarithmic average-case query and update times (skip lists) can be considered. These methods store the dictionary items at the nodes of a binary tree or of a graph and require $O(\log n)$ transfers in the worst case to perform a query or update operation).

To reduce the performance difference between internal-memory accesses and external-memory accesses for searching, dictionaries can be represented using a multi-way search tree. This approach gives rise to a generalization of the $(2,4)$ tree data structure to a structure known as the (a,b) tree.

(a, b) Trees

Formally, an (a, b) tree is a multi-way search tree such that each node has between a and b children and stores between a-1 and b-1 items, where a and b are integers. By setting the parameters a and b appropriately with respect to the size of disk blocks, a data structure can be derived that achieves good external-memory performance.

An (a, b) tree, where a and b are integers, such that $2 \leq a \leq (b+1)/2$, is a multi-way search tree T with the following additional restrictions:

→ Size property: Each internal node has atleast a children, unless it is the root, and has atmost b children.

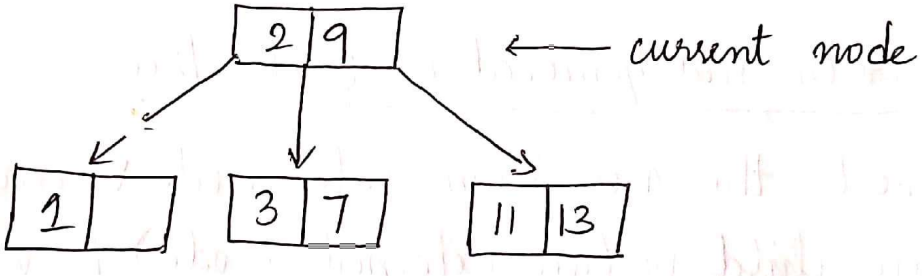
→ Depth property: All the external nodes have the same depth.

Searching in an (a, b) Tree – If T is an (a, b) tree, then D(v) stores at most b items. Let f(b) denote the time for performing a search in a D(v) dictionary. Searching in an

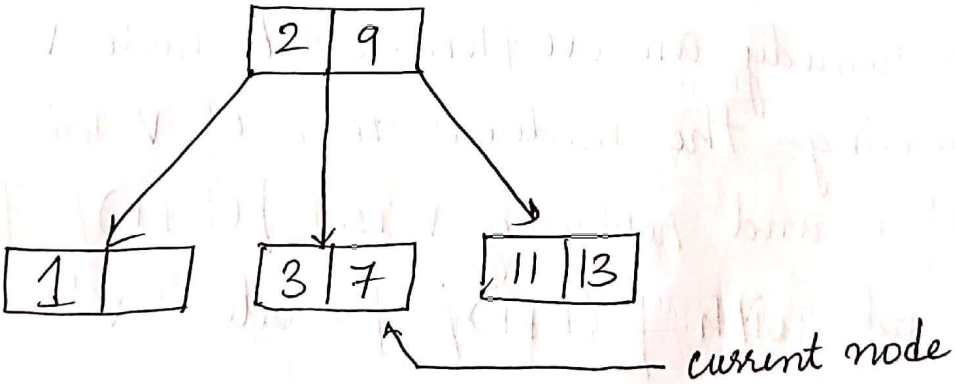
an underflow, either perform a transfer with a sibling of v that is not an a-node or perform a fusion of v with a sibling that is an a-node. The new node w resulting from the fusion is a (2a-1)-node.

Example :

Search 5 in the following 2-3 tree



$2 < 5 < 9$ → Search middle node



Element 5 not found in tree.

Insert 4, 10, 1 in the following 2-3 tree

(a,b) tree T with n items takes

$$O\left(\frac{f(b)}{\log a}\log n\right)$$

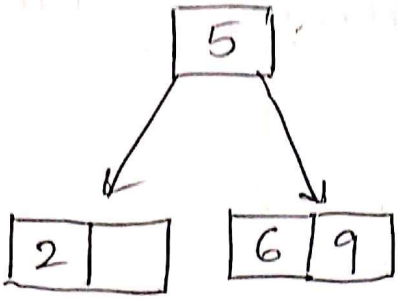
If 'b' is a constant and thus 'a' is also, then the search time is $O(\log n)$, independant of the specific implementation of secondary structures.

→ The main applications of (a,b) trees is for dictionaries stored in external memory.

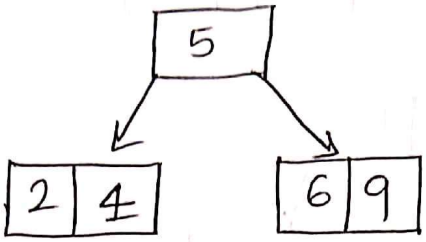
Insertion and Removal in (a,b) Tree –

Insert the new item into node v and add a new child w (an external node) to v. An overflow occurs when an item is inserted into a b-node v, which becomes an illegal (b+1) node. To remedy an overflow, split node v by moving the median item of v into the parent of v and replacing v with $\lceil (b+1)/2 \rceil$ node - v' and with $\lfloor (b+1)/2 \rfloor$ node - v''.

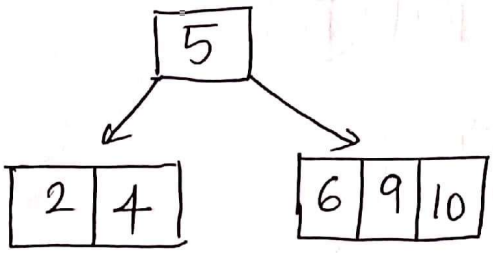
Removing a item from a node v - external node is removed always. An underflow occurs when a key is removed from an a-node v, distinct from the root, which causes v to become an illegal (a-1) - node. To remedy

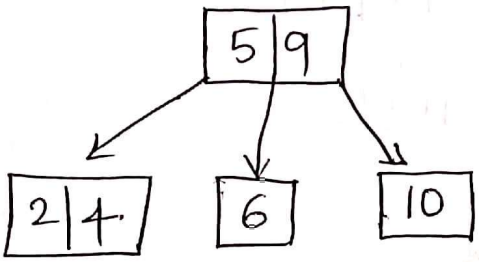


Insert 4 →

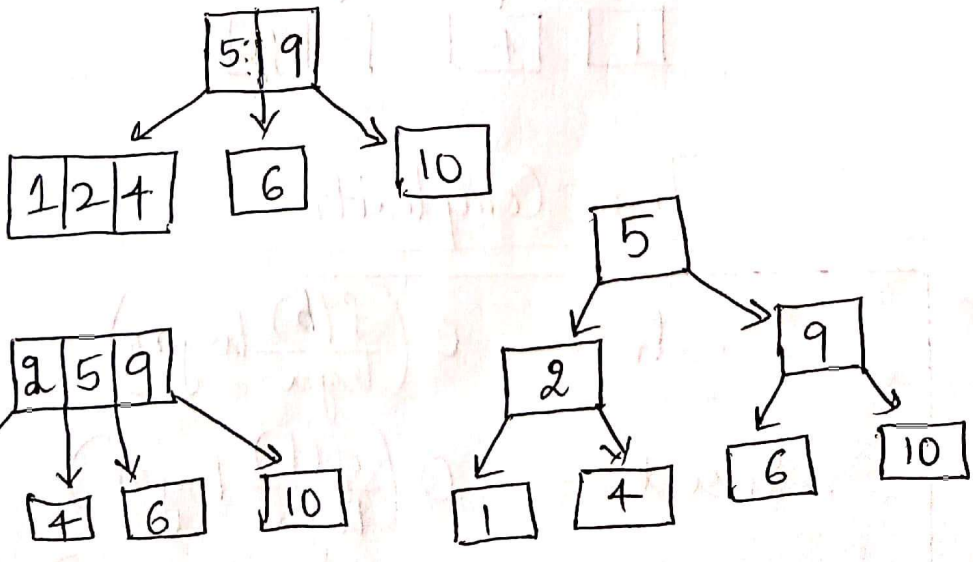


Insert 10 →

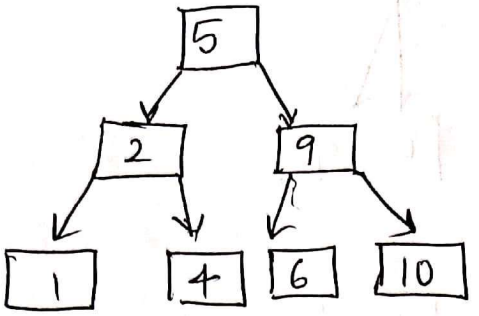




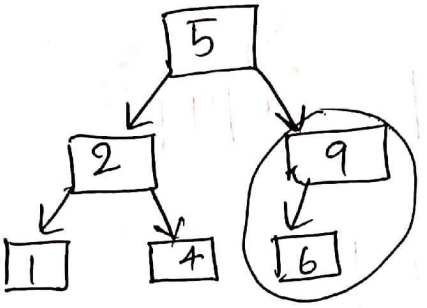
Insert 1 →



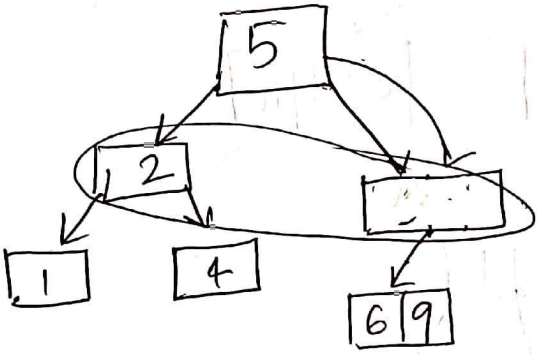
Delete 10 in the following 2-3 tree

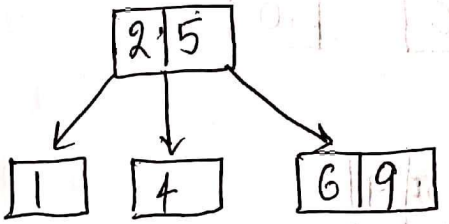


Delete 10 →



Merge & fuse





Time Complexities

Search — $O\left(\frac{f(b)}{\log a} \log n\right)$

Insert — $O\left(\frac{g(b)}{\log a} \log n\right)$

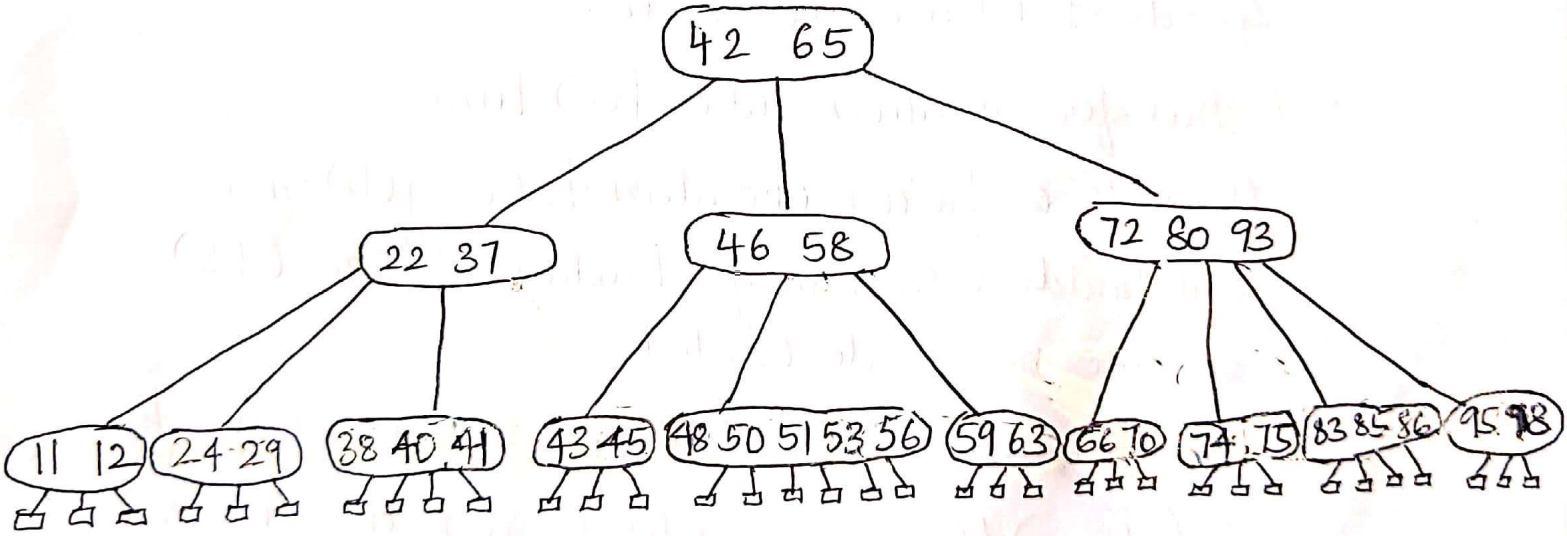
Remove — $O\left(\frac{g(b)}{\log a} \log n\right)$

B-Trees

A specialized version of the (a, b) tree data structure, which is an efficient method for maintaining a dictionary in external memory, is the data structure known as the "B-tree".

A B-tree of order 'd' is simply an (a, b) tree with $a = \lceil d/2 \rceil$ and $b = d$.

Example: A B-tree of order 6



→ Parameterizing B-trees for External Memory: In B-trees, 'd' can be choosen so that the 'd' children references and the 'd-1' keys stored at a node can all fit into a single disk block. Assume d is $\Theta(B)$, a and b are $\Theta(B)$ and $f(b) = c$ and $g(b) = c$ for some constant

The complexities are based on the following assumptions and facts :

- The (a,b) tree T is represented using the data structure, and the secondary structure of the nodes of T support search in $f(b)$ time, and split and fusion operations in $g(b)$ time
- The height of an (a,b) tree storing n elements is at most $O((\log n)/\log a))$
- A search visits $O((\log n)/(\log a))$ nodes on a path between the root and an external node, and spends $f(b)$ time per node.
- A transfer operation takes $f(b)$ time.
- A split or fusion operation takes $g(b)$ time and builds a secondary structure of size $O(b)$ for the new node created.
- An insertion or removal of an element visits $O((\log n)/(\log a))$ nodes on a path between the root and an external node, and spends $g(b)$ time per node.

→ An (a,b) tree implements an n-item dictionary to support performing insertions and removals in $O((g(b)/\log a)\log n)$ time, and performing search/find queries in $O((f(b)/\log a)\log n)$ time.

$c \geq 1$, for each time a node is accessed to perform a search or an update operation, only a single disk transfer is performed.
Therefore, any dictionary search or update operation on a B-tree requires only

$$O(\log_{\lceil d/2 \rceil} n) = O(\log n / \log B)$$
$$= O(\log_B n)$$

disk transfers.

→ An insert operation proceeds down the B-tree to locate the node in which the new item is to be inserted. If the node would overflow because of this insertion, then this node is split into two nodes that have $\lfloor (d+1)/2 \rfloor$ and $\lceil (d+1)/2 \rceil$ children, respectively. This process is then repeated at the next level up, and will continue for at most $O(\log_B n)$ levels.

→ In a remove operation, remove an item from a node, and if this results in a node underflow, move references from a sibling node with atleast $\lceil d/2 \rceil + 1$ children or

perform a fusion operation of this node with its sibling. This will continue up the B-tree for atmost $O(\log_B n)$ levels.

→ A B-tree with n items executes $O(\log_B n)$ disk transfers in a search or update operation, where B is the number of items that can fit in one block.

→ The requirement that each internal node have atleast $\lceil d/2 \rceil$ children implies that each disk block used to support a B-tree is atleast half full.